



ONLY GAMES

Presented by:

Preuttipong Rimchaisith 6610755370

Apivit Denpruktham 6610755354

Nattaporn Wilaisil 6610750397

Chanoudom Tann 6610755487

Nawat Sattayaporn 6610755362

Firas Chalghaf 6810930138



TABLE OF CONTENT

Introduction

Project Scope

Main User

Unique Features

System Overview

ER diagram

Relational Schema

Normalization

Example Queries

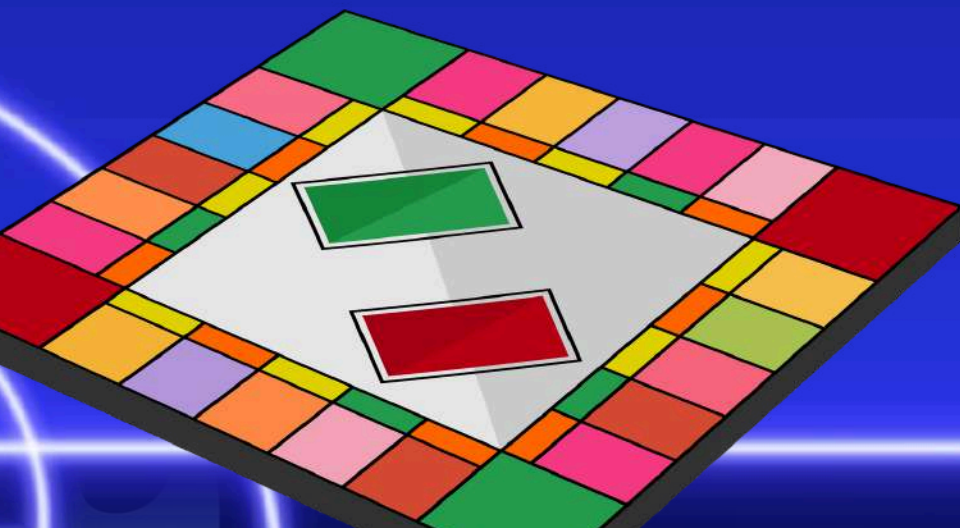
Design Decision

INTRODUCTION

- Gaming platforms today mainly focus on buying games
- Limited flexibility after purchase
- Users cannot easily share or transfer games

👉 OnlyGames addresses this by:

- Allowing users to buy and exchange games
- Supporting reviews and user interaction
- Providing a more flexible gaming experience



WHAT MAKES US DIFFERENT?

- Similar to platforms like Steam and Epic Games
- BUT focuses on flexibility and user control

↺ Game Exchange

- Transfer owned games between users
- Reduces waste and increases game usage
- 👉 Gives users more freedom in how they access games

★ Future approach

- Game renting feature



PROJECT SCOPE

Included:

- Game purchasing system
- Game exchange feature
- User reviews

Not Included:

- Real-time chat
- Payment gateway
- Game download system

Assumptions:

- Exchange is allowed only if the developer permits it.
- Orders record all past purchases and never change.
- UserLibrary shows the current games a user can access, which may change over time.



MAIN USER

System Users

Content:

- Users (players)
 - Buy games
 - Exchange games
 - Write reviews
- Admin
 - Manage games
 - Manage categories



SYSTEM OVERVIEW

The OnlyGames system is a relational database platform that manages:

User Management

- User profiles and authentication
- Wallet and account information

Game Management

- Games, categories, developers, publishers
- Tags, system requirements, and discounts

Transaction System

- Orders and order details
- Cart and wishlist functionality

Ownership & Access

- UserLibrary for current owned games
- Supports dynamic ownership changes

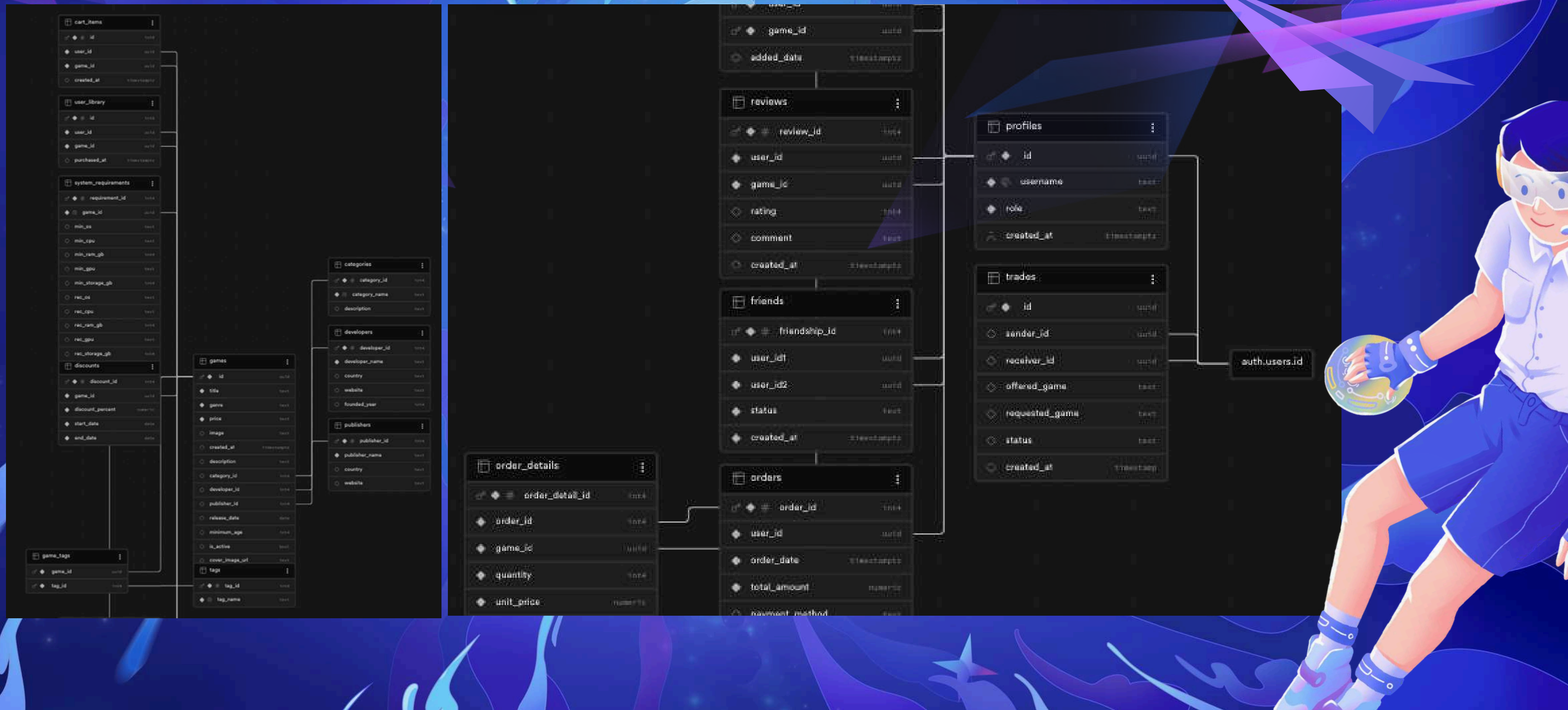
Interaction Features

- Reviews and ratings
- Friend system

Unique Features

- Game exchange (ownership transfer between users)

ER DIAGRAM



ER DIAGRAM

Main Relationships:

- Profiles 1:M Orders
- Orders 1:M OrderDetails
- Games 1:M OrderDetails
- Categories 1:M Games
- Developers 1:M Games
- Publishers 1:M Games
- Profiles 1:M Reviews
- Games 1:M Reviews
- Profiles 1:M UserLibrary
- Games 1:M UserLibrary
- Games M:N Tags through GameTags

Unique Feature Relationships:

Game Exchange / Trades

- Profiles 1:M Trades as sender
- Profiles 1:M Trades as receiver
- Games 1:M Trades as offered game
- Games 1:M Trades as requested game

This ERD shows how the main entities connect through primary keys and foreign keys. Most relationships are one-to-many, such as one profile having many orders, reviews, library records, and cart items. Many-to-many relationships are handled using bridge tables, such as game_tags connecting games and tags, and order_details connecting orders and games.



RELATIONAL SCHEMA WITH PK/FK

How our database tables are structured using Primary Keys (PK) and Foreign Keys (FK):

- profiles(id PK, username, email, role)
- games(id PK, title, genre, price, category_id FK, developer_id FK, publisher_id FK, exchange_allowed)
- categories(category_id PK, category_name)
- developers(developer_id PK, developer_name)
- publishers(publisher_id PK, publisher_name)
- orders(order_id PK, user_id FK, order_date, total_amount, status)
- order_details(order_detail_id PK, order_id FK, game_id FK, quantity, unit_price)
- user_library(id PK, user_id FK, game_id FK, purchased_at)
- reviews(review_id PK, user_id FK, game_id FK, rating, comment)
- cart_items(id PK, user_id FK, game_id FK)
- wishlists(user_id PK/FK, game_id PK/FK)
- tags(tag_id PK, tag_name)
- game_tags(game_id PK/FK, tag_id PK/FK)
- trades(trade_id PK, sender_id FK, receiver_id FK, offered_game_id FK, requested_game_id FK, status)
- discounts(discount_id PK, game_id FK, discount_percent, start_date, end_date)
- friends(friendship_id PK, user_id1 FK, user_id2 FK, status)
- system_requirements(requirement_id PK, game_id FK)
- wallets(user_id PK/FK, balance)



NORMALIZATION

Our database follows Third Normal Form (3NF)

- 1NF: Each column stores atomic (single) values
- 2NF: Non-key attributes depend on the whole primary key
- 3NF: Non-key attributes do not depend on other non-key attributes

📌 Examples:

- Category data is stored in Categories, not repeated in Games
- Developer data is stored in Developers, not repeated in Games
- Purchase history is separated from current access using
- Orders and UserLibrary

🎯 Why this matters:

- Reduces data redundancy
- Prevents update anomalies
- Ensures data consistency



SQL Implementation

- Database implemented using PostgreSQL on Supabase
- Tables created using CREATE TABLE
- Primary keys and foreign keys defined
- Constraints used:
 - NOT NULL
 - UNIQUE
 - CHECK
- Relationships enforced using Foreign Keys

Full SQL script is included in the project report



SQL QUERY DEMONSTRATION

Games with Category:

- Join Games and Categories to show game details

User Purchase History:

- Show all orders and purchased games per user

Total Spending per User:

- Calculate total spending using SUM()

Top 3 Popular Games:

- Find most purchased games using COUNT()

Game Exchange Requests:

- Show pending trades between users



Design Decisions

- Used OrderDetails to support orders with multiple games
 - to distinguish purchase history and current access
- Implemented Trades (Game Exchange)
 - to allow ownership transfer between users
- Added developer control for exchange
 - games can only be exchanged if allowed
- Used Foreign Keys
 - to maintain data consistency and relationships





ADVANTAGES

- Flexible system supporting buy and exchange
- Separation of Orders and UserLibrary
 - → Enables dynamic ownership and access control
- Developer control over exchange
 - → Ensures realistic and controlled system behavior
- Well-structured relational design
 - → Scalable and easy to maintain
- Reflects real-world application scenarios



LIMITATIONS

- No real-time system
- No external payment gateway integration
 - → payment data is stored only for database simulation
- Simplified exchange logic
 - → does not handle complex trade scenarios



conclusion

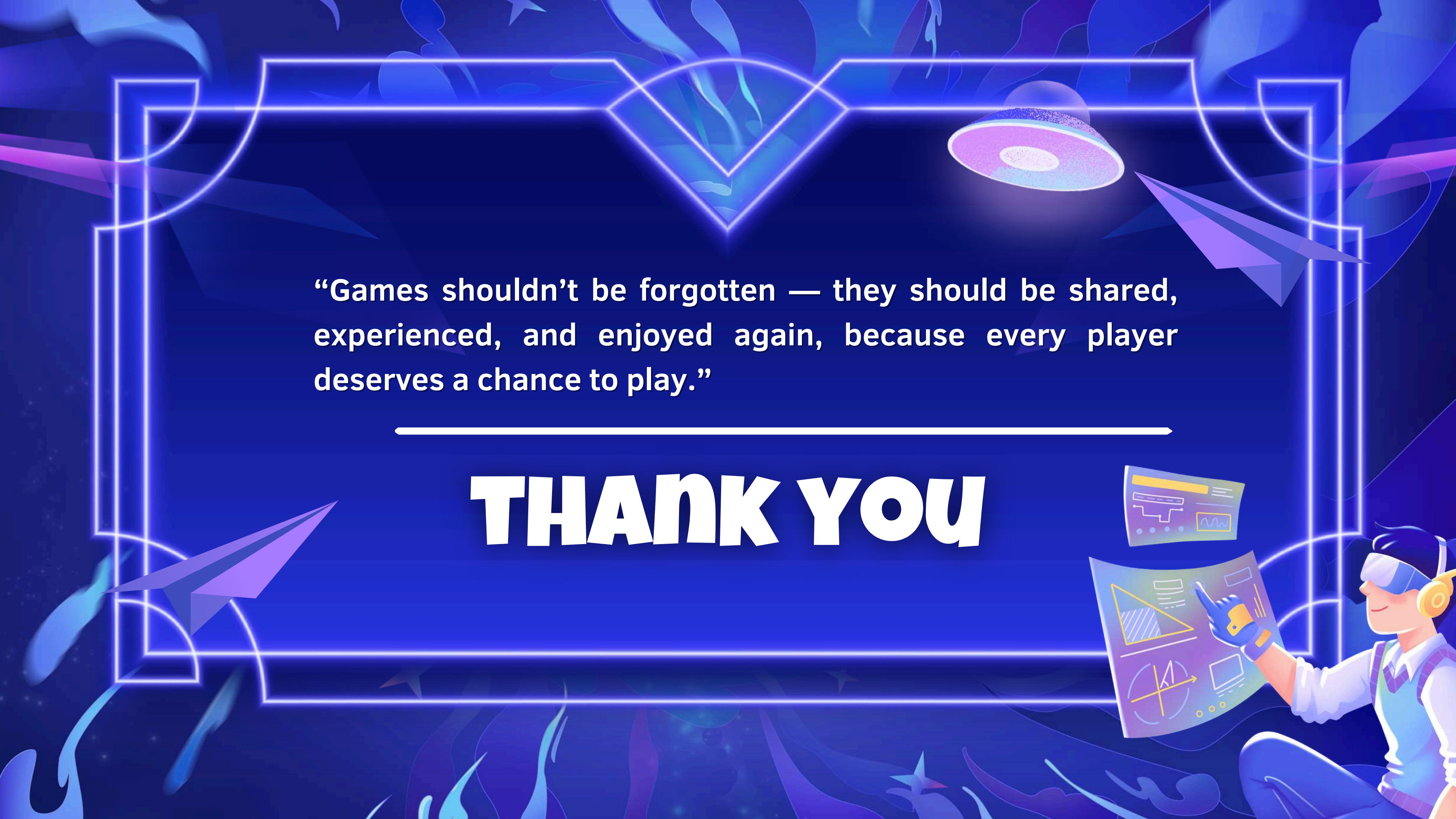
In conclusion, we successfully designed and implemented a relational database system for a game platform that supports core functionalities such as transactions, ownership tracking, and user interactions.

Beyond the technical design, we focused on creating a system that provides more flexibility for users through features like game exchange.

These features allow games that are no longer actively played to be used again, instead of being left unused. At the same time, they make gaming more accessible for users who may not have the budget to purchase every game.

Even though this is a database project, we believe it reflects an idea that could be further developed into a real-world system.





“Games shouldn’t be forgotten — they should be shared, experienced, and enjoyed again, because every player deserves a chance to play.”

THANK YOU